



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

Escalando la síntesis composicional de controladores mediante una nueva técnica de minimización para branching bisimilarity

Tesis de Licenciatura en Ciencias de la Computación

Cindy Eliana Levi

Director: Hernán Gabriel Gagliardi

Codirector: Sebastian Uchitel

Buenos Aires, 2026

ESCALANDO LA SÍNTESIS COMPOSICIONAL DE CONTROLADORES MEDIANTE UNA NUEVA TÉCNICA DE MINIMIZACIÓN PARA BRANCHING BISIMILARITY

Esta tesis presenta la definición y análisis exploratorio sobre las fases que están involucradas en el algoritmo *on-the-fly* de síntesis de directores para resolver problemas de Control de Eventos Discretos con la propiedad central de tipo non-blocking. Se busca de esta manera encontrar posibilidades de mejora en la heurística RA, la cual logra actualmente buenos resultados frente a heurísticas tradicionales. Luego de realizada la exploración y análisis inicial sobre el comportamiento de las etapas en la exploración, implementamos algunos cambios sobre la heurística RA para poder experimentar y analizar los resultados. De esa manera realizamos una propuesta de mejora para fortalecer las etapas de exploración analizadas previamente y lograr mejor *performance*.

Palabras claves: Sistemas de Eventos Discretos, Síntesis de Controladores, Control Dirigido, Control Supervisado, Síntesis on-the-fly, LTS, Non-Blocking.

AGRADECIMIENTOS

A Silvana y Beto, por criarme y acompañarme en cada paso con infinito amor.

A Mariana, Laura, Cecilia y Mariano, porque siempre me hicieron sentir querido, protegido y ayudado.

A la casa de Perón, porque en comunidad transcurrió gran parte de la carrera y en donde Milagros, Ale, Julls, Alicia, Franco, Tito, Martín, Cabeza y Virginia me bancaron las jornadas de estudio y siguieron cada uno de mis exámenes.

A Fernando, Pablo, Santiago y Juan Pablo por el ejercicio incesante de la amistad y por ser un ejemplo a seguir dentro y fuera de la facultad.

A Fabio y a Roni, porque hicimos codo a codo cada examen, final y TP de la carrera.

A Daniela, que en muchos momentos difíciles de la carrera supo ayudarme y convencerme de seguir.

Al Departamento de Computación y toda la gente que lo compone por ser una segunda casa desde hace tantos años. Al LaFHIS y todxs sus integrantes por haberme recibido en este último tramo con los brazos abiertos y haberme apoyado en todo momento.

A mi director de Tesis Sebastian Uchitel y a mi co-director Sebastian Zudaire por la guía y por la inmensa generosidad con la que me acompañaron a lo largo de este trabajo.

A la militancia de Identidad, porque aprender de ellxs que vale la pena organizarse, luchar y trabajar por una facultad y un país mejor es una de las cosas más valiosas que me dejaron estos años. A Juliana y Abril, que me permiten irme del claustro con el inmenso orgullo de haber votado y acompañado a las primeras dos Presidentas Peronistas del CECEN.

A Julia porque construimos juntxs, desde el amor, los días más felices.

Índice general

1..	Introducción	1
2..	Antecedentes	5
2.1.	Definiciones utilizadas	5
2.1.1.	Autómatas y composición paralela	5
2.1.2.	Controlador	6
2.2.	Un ejemplo concreto	7
3..	Branching bisimilarity	9
3.1.	Definición formal	9
3.2.	WSOE: definición formal	9
3.3.	Diferencias intuitivas y casos motivadores	9
3.4.	Teorema de equivalencia y demostración	9
3.5.	Corolarios	9
3.6.	Relación con otras equivalencias	9
4..	El algoritmo de Groote et al.	11
4.1.	Algoritmo $O(m \log n)$ de Groote et al.	11
5..	Implementación del algoritmo	13
5.1.	Introducción	13
5.2.	Primera implementación del algoritmo	14
5.3.	Primeros tests	17
5.3.1.	Integración con el algoritmo composicional	17
5.4.	Primeras optimizaciones	17
5.4.1.	Cambio de estrategia: dos fases que alternan	17
5.4.2.	Implementación de nuevas estructuras	19
5.5.	Alcanzando la complejidad teorica	19
5.5.1.	Refinable Partitions	19
5.5.2.	Linked Transition Partitions	23
5.5.3.	El método split como corrutinas	26
6..	Validación y experimentación	27
6.1.	Metodología	27
6.2.	Validación de correctitud	28
6.3.	Resultados	28
6.3.1.	Tamaño de la reducción final	28
6.3.2.	Tiempos	28
6.3.3.	Memoria	29
6.4.	Discusión	31
7..	Conclusiones	33

8.. Apéndice	35
------------------------	----

1. INTRODUCCIÓN

*Lo que vieron mis ojos fue simultáneo:
lo que transcribiré, sucesivo,
porque el lenguaje lo es.
Algo, sin embargo, recogeré.
—JLB*

La automatización de tareas ha sido a lo largo de la historia un desafío *motivante* para las Ciencias de la Computación. Desde sus inicios se buscó la manera de automatizar las tareas tediosas o repetitivas. Yendo un poco más lejos, la posibilidad de una computadora que *se programe* a sí misma es abordada por distintas ramas de la computación. Entre ellas se encuentran muchas agrupadas dentro del término *Inteligencia Artificial*. En el trabajo que presentamos nos abocaremos al caso particular de programas que generan **Síntesis Automática de Controladores**.

En este área de estudio buscamos desarrollar código que **sintetice** una solución de un determinado problema de forma **automática** partiendo de ciertas *reglas y objetivos* (o pre/post condiciones) que se proveen y es necesario cumplirlos. De esta manera, lo que devuelve nuestro programa es una estrategia que garantiza alcanzar una solución, si es que existe. Por otro lado, en el caso de que dadas las reglas y los objetivos planteados no exista una solución posible, avisa sobre la inexistencia de la misma. En el caso de garantizar la existencia de una solución, a la estrategia que permite alcanzarla se la conoce con el nombre de **Controlador**.

Este tipo de programas tienen múltiples aplicaciones, entre ellas podemos nombrar el control de aviones para demarcar incendios forestales [4], donde a veces incluso es necesario que el programa pueda adaptarse en medio de su ejecución.

El problema de *síntesis* fue estudiado dentro de distintas áreas como: Control de Eventos Discretos [5], Síntesis Reactiva [6] y Planning automático [7]. Si bien este trabajo se basa en resultados que provienen de combinar enfoques de las tres áreas, nos basamos principalmente en Control de Eventos Discretos (Discrete Event Control o DEC).

En este enfoque modelamos el problema utilizando autómatas finitos, también conocidos como máquinas de estados finitas. Un autómata finito, está conformado por estados y transiciones entre ellos. Podemos pensar los *estados* como puntos y las transiciones como flechas de un solo sentido que unen a uno o varios de ellos. A estos puntos los llamamos estados ya que representan un momento particular del dominio que se quiere modelar, por ejemplo en el caso del avión mencionado anteriormente se podría hablar de estados que representen posibilidades como: *avión aterrizado*, *avión sin combustible*, *avión en vuelo*, etc.

Por otro lado, vamos a tomar dentro de nuestro modelo a las transiciones como posibles acciones. En este caso algunas transiciones las podemos controlar y otras estarán fuera de nuestro alcance. Volviendo al ejemplo del avión, una acción **controlable** podría ser *encendido de motor* mientras que la acción *aterrizaje de emergencia* probablemente sea representada en nuestro modelo como una acción **no controlable**.

En el autómata mencionado además existen estados distinguidos. Por un lado tomaremos un estado como *inicial* y será el estado desde el cual empezaremos la exploración.

Además habrá *estados objetivos* que serán los estados a los cuales buscaremos llegar. En este caso de análisis y a lo largo de la tesis llamaremos *marcados* a estos estados objetivo y buscaremos que el sistema no solo pueda alcanzarlos, sino que sea capaz de hacerlo infinitas veces y de manera segura. Esta última condición de *seguridad* se refiere intuitivamente a que debemos llegar a un estado marcado mediante una secuencia de acciones que evite estados no controlables no deseados en el sistema. A esto se le agrega la condición de llegar al estado marcado infinitas veces, lo que refiere a que buscaremos poder mantenernos *de forma controlable* con transiciones que nos garanticen un ciclo de permanencia en el estado marcado.

El valor de este enfoque y de un algoritmo que permita realizar síntesis es la generalización a distintos problemas. El autómata descrito puede modelar un avión como el caso mencionado, o el funcionamiento de una agencia de viajes como se verá en otro ejemplo o podrá modelar el funcionamiento de diversos problemas complejos. De todas formas, nuestro algoritmo podrá generar un **controlador** sin necesitar ningún cambio específico. Para lograr esto trabajará un experto en el área, quien decide las reglas y objetivos que conformarán el autómata a analizar, por ejemplo partir de un avión en un hangar e intentando llegar a un estado marcado que describa a un avión aterrizado luego de formar un patrón en el cielo. Pero como podemos ver, este experto no debe preocuparse por como se sintetiza el controlador que resuelve el autómata que modela su problema, esa parte está garantizada por el algoritmo de síntesis que genera el controlador.

Por último cabe destacar que el modelado de un sistema complejo resulta prácticamente imposible utilizando un solo autómata. Por esta razón es que se suelen modelar distintas partes del problema original como autómatas independientes y luego se componen para formar el modelo completo del sistema. Esta composición a la que nos referimos debe respetar ciertas reglas y garantiza que un estado del sistema compuesto representa la combinación de estados en que está cada componente al finalizar.

Tradicionalmente la forma de resolver el problema consiste en tomar estos autómatas independientes, realizar la composición y obtener el autómata completo para que sea analizado por el programa para sintetizar el controlador. Sin embargo, cuando el problema a resolver escala en tamaño de estados y transiciones de forma abrupta el autómata compuesto se vuelve demasiado grande, haciendo imposible su análisis y en ocasiones resultando imposible también el proceso de composición.

En este contexto surge el enfoque de exploración *on-the-fly*, en donde la composición se construye a medida que se van explorando transiciones mediante las cuales intentamos resolver el problema de síntesis evitando, en la medida de lo posible, la composición de todo el modelo. Este enfoque fue presentado por primera vez en la tesis doctoral de Daniel Ciolek [1] y luego profundizado en la tesis de Licenciatura de Matías Durán y Florencia Zanollo [2] cuyo trabajo resulto publicado [3].

Basándonos en este trabajo, en esta tesis analizaremos el comportamiento del algoritmo de exploración del autómata compuesto que describimos anteriormente. Vamos a definir y describir ciertas fases o etapas relevantes del proceso, buscando experimentar el desempeño de las heurísticas existentes y que posibilidades de mejora existen en base a esta información.

En el capítulo 2 presentaremos los antecedentes y definiciones necesarias alrededor del problema de síntesis composicional de controladores: LTS y composición paralela, el problema de control *safe* y *non-blocking*, y la síntesis composicional.

En el capítulo 3 introducimos *branching bisimilarity*, damos la definición formal de

WSOE y demostramos el teorema de equivalencia entre ambas —el aporte teórico que justifica reemplazar una por la otra—, ubicándola además frente a las equivalencias clásicas.

En el capítulo 4 presentamos el algoritmo $O(m \log n)$ de Groote et al. [9] que calcula branching bisimilarity de forma eficiente. Luego, en el capítulo 5, detallamos su adaptación e implementación dentro del flujo composicional de MTSA [8].

En el capítulo 6 presentamos la experimentación, comparando la minimización por WSOE contra branching bisimilarity en tiempo, memoria y tamaño de los LTS minimizados. Finalmente, en el capítulo 7 exponemos las conclusiones y aportes del trabajo.

2. ANTECEDENTES

En este capítulo agregaremos algunas definiciones y explicaciones necesarias para poder avanzar con la definición y análisis formal del problema composicional de síntesis de controladores.

2.1. Definiciones utilizadas

2.1.1. Autómatas y composición paralela

Definición 1 (*Autómata Determinístico*): Un autómata determinístico es una tupla $T = (S_T, A_T, \rightarrow_T, \bar{t}, M_T)$, donde: S_T es un *conjunto finito de estados*, A_T es un *conjunto finito de eventos del autómata*, $\rightarrow_T: S_T \times A_T \rightarrow S_T$ es una *función de transición*, $\bar{t} \in S_T$ es el *estado inicial del autómata* y $M_T \subseteq S_T$ es el conjunto de *estados marcados*.

Notación 1 (*Pasos y corridas*): Notamos $(t, \ell, t') \in \rightarrow_T$ como $t \xrightarrow{\ell}_T t'$ y lo llamamos un *paso*. A su vez, una *corrida* de una palabra $w = \ell_0, \dots, \ell_k$ en T , es una secuencia de pasos tal que $t_i \xrightarrow{\ell_i}_T t_{i+1}$ para todo $0 \leq i \leq k$, notado como $t_0 \xrightarrow{w} \dots \rightarrow_T t_{k+1}$.

Notación 2: Decimos que un estado s es alcanzado por una palabra w en una corrida comenzando en el estado s' , anotado como $s \in w(s')$, cuando $\exists w_0 \cdot \exists w_1 \cdot w = w_0 \cdot w_1$ y $s' \xrightarrow{w_0} \dots \rightarrow_E s$.

Los autómatas definen un lenguaje, un conjunto de palabras que aceptan. Dado un conjunto de eventos A , notamos con A^* al conjunto de palabras finitas de eventos de A . El lenguaje generado por un autómata T , notado como $\mathcal{L}(T)$, es el conjunto de palabras formadas por eventos que cumplen $\rightarrow_T$. Formalmente, si $w \in A_T^*$, entonces $w \in \mathcal{L}(T)$ si y sólo si existe una corrida para w comenzando desde el estado inicial $\bar{t}$ de T , que notamos $\bar{t} \xrightarrow{w} \dots \rightarrow_T t'$. Generalmente los estados marcados se utilizan para indicar el logro de una tarea. Por lo tanto, consideramos como lenguaje marcado (*o aceptado*) por T (lo cual notamos $\mathcal{L}_m(T)$) al conjunto de palabras cuya corrida termina en un estado marcado.

Formalmente, sea $w \in \mathcal{L}(T)$, entonces $w \in \mathcal{L}_m(T)$ si y solo si hay una corrida de w que comienza en el estado inicial $\bar{t}$ y alcanza un estado marcado $t_m \in M_T$, que notamos $\bar{t} \xrightarrow{w} \dots \rightarrow_T t_m$. Este último concepto es relevante para la definición de la composición paralela. Es una parte fundamental del trabajo para definir las palabras aceptadas por el problema *nonblocking* ya que el lenguaje aceptado por la planta compuesta es el mismo que el aceptado por cada uno de los componentes por separado.

Definición 2 (*Composición Paralela*): La composición paralela ($\parallel$) de dos autómatas T y Q es un operador simétrico y asociativo que produce un autómata $T \parallel Q = (S_T \times S_Q, A_T \cup A_Q, \rightarrow_{T \parallel Q}, \langle \bar{t}, \bar{q} \rangle, M_T \times M_Q)$, donde $\rightarrow_{T \parallel Q}$ es la menor relación que satisface las siguientes reglas (omitimos la versión simétrica de la primera regla):

$$\frac{t \xrightarrow{\ell}_T t'}{\langle t, q \rangle \xrightarrow{\ell}_{T \parallel Q} \langle t', q \rangle} \ell \in A_T \setminus A_Q \qquad \frac{t \xrightarrow{\ell}_T t' \quad q \xrightarrow{\ell}_Q q'}{\langle t, q \rangle \xrightarrow{\ell}_{T \parallel Q} \langle t', q' \rangle} \ell \in A_T \cap A_Q$$

Hay ciertas características de la composición paralela a destacar. En primer lugar, la última regla realiza la sincronización entre componentes. Segundo, que el número de

estados en $T_0 \parallel \dots T_n$ puede crecer exponencialmente con respecto al número de autómatas a componer. Tercero, que el lenguaje aceptado por la composición contiene las palabras que alcanzan estados marcados en todos los componentes simultáneamente.

2.1.2. Controlador

Dado un (conjunto de) autómatas y una partición de sus eventos en dos subconjuntos (controlables y no controlables), lo que buscamos es un *controlador* que limite el vocabulario aceptado por el autómata de forma tal de mantener un camino posible a los estados marcados del mismo.

Un controlador observa las transiciones no controlables y puede deshabilitar algunas transiciones controlables para generar una planta restringida. Una palabra w pertenece al lenguaje generado por T restringido por una función del controlador $\sigma : A_T^* \mapsto 2^{A_T}$ (anotado como $\mathcal{L}^\sigma(T)$) si cada prefijo de w sobrevive a σ . Formalmente, sea $w = \ell_0, \dots, \ell_k$ una palabra en $\mathcal{L}(T)$, entonces $w \in \mathcal{L}^\sigma(T)$ si y solo si para todo $0 \leq i \leq k : \bar{t}^{\ell_0, \dots, \ell_i} \xrightarrow{T} t_{i+1} \wedge \ell_i \in \sigma(\ell_0, \dots, \ell_{i-1})$.

Definición 3 (*Problema de control Safe y Non-Blocking*): Un problema de control con objetivos **Safe** y **Non-Blocking** composicional es una tupla $\mathcal{E} = (E, A_E^C, S_E^I)$, donde E es un conjunto de autómatas $\{E_0, \dots, E_n\}$. En nuestro caso haremos un abuso de notación y usaremos $E = (S_E, A_E, \rightarrow_E, \bar{e}, M_E)$ para referirnos a la composición $E_0 \parallel \dots \parallel E_n$. Por otro lado, $A_E^C \subseteq A_E$ es el conjunto de eventos controlables y definimos análogamente a $A_E^U = A_E \setminus A_E^C$ como el conjunto de eventos **no** controlables.

Una solución para $\mathcal{E}$ es un supervisor $\sigma : A_E^* \mapsto 2^{A_E}$, tal que σ es:

- *Controlable*: $A_E^U \subseteq \sigma(w)$ con $w \in A_E^*$
- *Safe y non-blocking*: para cada palabra $w \in \mathcal{L}^\sigma(E)$ existe una palabra no vacía $w' \in A_E^*$ tal que la concatenación $ww' \in \mathcal{L}^\sigma(E)$ y $\bar{e} \xrightarrow{ww'}_E e_m$ con $e_m \in M_E$ (un estado marcado de E).

Por lo tanto concluimos que un supervisor σ es *controlable* si deshabilita solamente eventos controlables, y es *safe y non-blocking* si podemos garantizar para cualquier palabra del lenguaje una extensión que le permita llegar a un estado marcado. A continuación definiremos como *ganadores* (*perdedores*) a los estados que podemos (no podemos) incluir en un controlador.

Decimos que un estado s es **ganador** en el problema $\mathcal{E} = (E, A_E^C, S_E^I)$ si hay una solución (E_s, A_E^C) donde E_s es el resultado de cambiar al estado inicial de E a s .

Podemos pensar en un controlador *non-blocking* como un jugador optimista que se encarga de no perder, y solo requiere conocer un futuro camino posible para llegar a un estado ganador. Es importante notar que como se busca que cualquier palabra sea extensible a otro estado marcado, lo que se busca es pasar por algún estado marcado infinitas veces. Es decir, un estado e marcado que tenga un camino para que el jugador pueda volver *controlablemente* al mismo estado e .

Por esto es que a lo largo de este trabajo analizaremos detenidamente la estructura de ciclo en nuestro algoritmo ya que sirve para deducir propiedades como que los primeros estados ganadores son aquellos que están en un loop controlable con un estado marcado dentro. Luego anotamos como ganador también a cualquier estado que controlablemente alcanza un estado ganador.

Los ciclos también son esenciales para encontrar los estados perdedores, ya que la única forma de que un estado sea perdedor es que no pueda alcanzar un estado ganador. En otras palabras, los estados perdedores son aquellos que forman parte de un loop que no tiene estados marcados ni transiciones salientes, así mismo los que llevan a un deadlock.

2.2. Un ejemplo concreto

Para ejemplificar y tener una noción acerca del tamaño que puede alcanzar el problema de composición y su posterior exploración, presentaremos una versión simplificada del problema **TravelAgency** que describiremos con mayor detalle posteriormente en el capítulo 6 y será usado para medir la *performance* del algoritmo.

En este problema se busca armar una agencia de venta online de paquetes vacacionales. El objetivo es que la misma orqueste los servicios (alquiler de auto, hotel, pasaje de avión, etc.) de forma automática de manera de obtener un paquete de vacaciones completo de ser posible, evitando pagar por paquetes incompletos.

Para cada servicio que se desea sub-contratar se consulta (query) si ese servicio está disponible. En caso de tener éxito y confirmar la disponibilidad, se espera a confirmar el resto para comprar todos al mismo tiempo. En caso de que algún otro servicio no esté disponible, se cancela la compra y se vuelve al estado inicial, lo cual no implica un gasto.

Este problema, descrito por ahora en términos generales, puede escalar de forma muy rápida si se incrementa la cantidad de servicios a contratar o la cantidad de pasos para reservar cada servicio, por lo que para este ejemplo ilustrativo asumimos que no hacen falta múltiples pasos para reservar un servicio, y solo mostramos el problema con 1 y 2 sub-servicios.

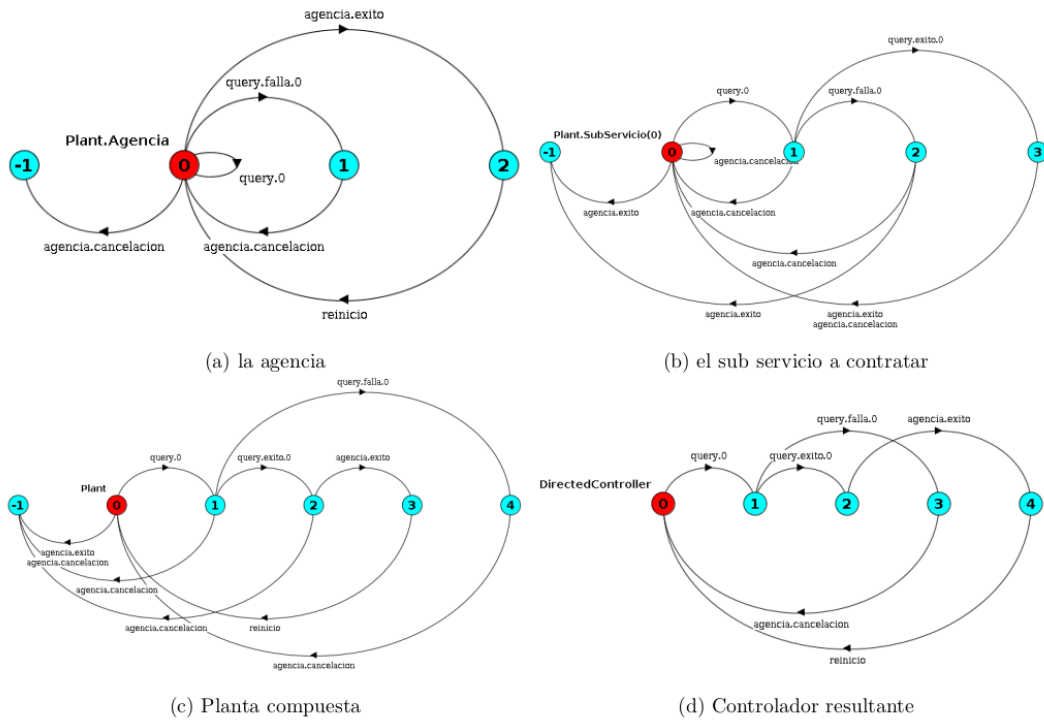


Fig. 2.1: Ejemplo del problema Travel Agency (TA) con un solo sub-servicio

Finalmente, 2.1d muestra un posible controlador para este problema. Destacamos que todos los eventos controlables que llevaban al estado -1 (el estado error), no son una posibilidad permitida por el controlador.

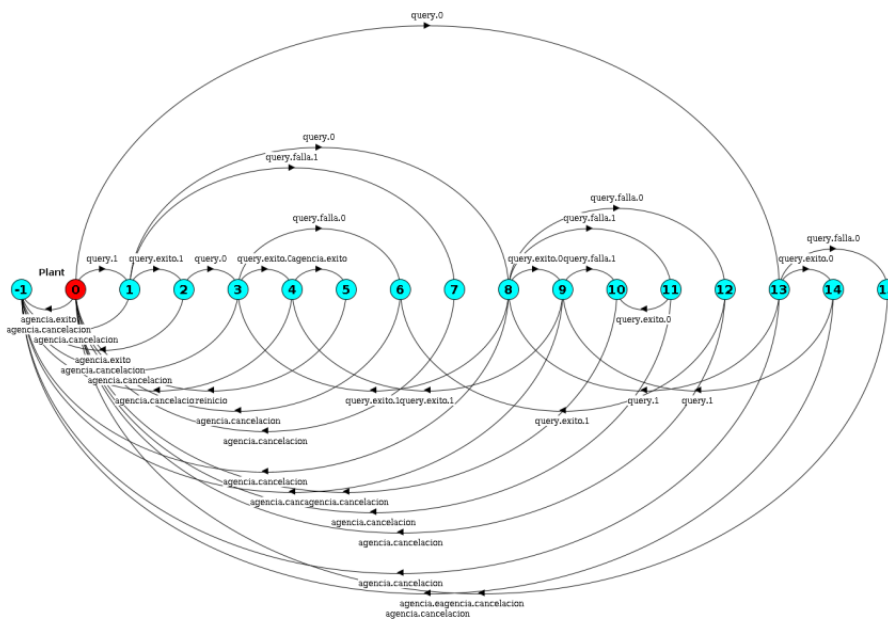


Fig. 2.2: Planta compuesta para 2 sub-servicios

Por esta razón reafirmamos el enfoque de exploración *on-the-fly* para resolver el problema de síntesis evitando, en la medida de lo posible, la composición de todo el modelo.

3. BRANCHING BISIMILARITY

- 3.1. Definición formal
- 3.2. WSOE: definición formal
- 3.3. Diferencias intuitivas y casos motivadores
- 3.4. Teorema de equivalencia y demostración
- 3.5. Corolarios
- 3.6. Relación con otras equivalencias

4. EL ALGORITMO DE GROOTE ET AL.

4.1. Algoritmo $O(m \log n)$ de Groote et al.

5. IMPLEMENTACIÓN DEL ALGORITMO

HG *⟨Está muy bien, lo único que me gustaría tener referencias al algoritmo que entiendo vas a poner en la sección 3 para no perderme tanto cuando leo⟩*

5.1. Introducción

La herramienta MTSA [8] es el *framework* de desarrollo realizado en Java para varias iniciativas de LaFHIS en el contexto de la investigación sobre modelado y verificación de sistemas concurrentes, así como síntesis de controladores. Es por eso que tanto el algoritmo composicional presentado por Uchitel y Gagliardi como la versión de WSOE utilizada para la composicionalidad fueron implementados en esta herramienta. Era entonces natural realizar el desarrollo del algoritmo de *branching bisimilarity* sobre el mismo *framework*. A su vez, nos permitirá comparar de forma más verídica ambos algoritmos, realizando pruebas de *benchmarks* tanto para tiempos de ejecución como para el uso de memoria. En primer lugar, se realizó la implementación del algoritmo de *branching bisimilarity* en el archivo `BranchingEquivalence.java`, creado en `mtstools/DCS/Compositional/` ya que bajo este módulo están las herramientas utilizadas por el algoritmo composicional.

agregar cita

A su vez, en `CompositionalApproach.java` se reemplazó la llamada original al algoritmo de WSOE por la llamada a la nueva función que utiliza el algoritmo de Groote y Jansen **HG** *⟨cita⟩*, reemplazando también los preprocesamientos necesarios para los algoritmos que se hacen en algunos casos fuera de las funciones principales.

La llamada al algoritmo de *branching bisimilarity* queda entonces de esta forma

```
Pair<Pi_s, Pi_t> partition
    = BranchingEquivalence.getPartitions(lts, alphabet,
    ↪ controllables, fluentGoals);

MTS<Long, String> minimizationResult =
    ↪ BranchingEquivalence.buildMinimisedMTSFromPartition(lts, tauLabels,
    ↪ controllables, partition);
```

donde en la función `getPartitions(...)` se implementa el algoritmo de *branching bisimilarity* de Groote y Jansen **HG** *⟨cita⟩* y se devuelve la partición final de estados y transiciones, y en `buildMinimisedMTSFromPartition(...)` se construye el MTS minimizado a partir de la partición calculada.

A su vez, el algoritmo recibe como parámetros el LTS en realidad a minimizar, el alfabeto local, el traductor total y los fluents objetivo. **HG** *⟨hay un par de estas cosas que habría que chequear si explicar antes que son o sino tenes que explicarlas acá, por ej. traductor total, fluents objetivo, etc.⟩*

En primer lugar, la modalidad de desarrollo fue implementar la lógica del algoritmo lo más fielmente al paper posible, es decir, sin tomar en cuenta las optimizaciones ni las cotas de complejidad. En la versión cero (v_0), el algoritmo replica la estructura del paper pero reemplazando las estructuras de datos propuestas por estructuras nativas de MTSA.

5.2. Primera implementación del algoritmo

Como fue explicado en el capítulo 3, el algoritmo de Groote y Jansen HG ⟨cita⟩ mantiene dos estructuras de refinamiento: una partición de estados Π_s y una partición de transiciones Π_t . En Π_s se encuentran los *bloques* de estados estables tal que si dos estados están en distintos bloques entonces **no** son *branching bisimilar*. En Π_t se encuentran los *bunches* de transiciones no-inertes, es decir, las transiciones que pueden distinguir estados. Ambas estructuras se refinan a la par: cada vez que un bloque de estados se parte en dos, se actualizan los bunches para reflejar el nuevo bloque, y cada vez que un bunch se parte en slices por (acción, bloque destino), se generan nuevos splitters para estabilizar los bloques afectados. El proceso continúa hasta llegar al punto fijo donde no quedan ni splitters pendientes ni bunches por refinar: las clases de equivalencia son entonces los bloques finales de Π_s .

La idea para esta primera implementación fue mantener la estructura del paper, donde el *loop* principal itera sobre cada conjunto de transiciones no triviales que están en Π_t , tomando un subconjunto chico de transiciones. Este subconjunto de transiciones, es separado de su bunch original y añadido a Π_t como un nuevo bunch. Luego, se deben refinar los bloques de Π_s para que queden estables respecto al nuevo conjunto de transiciones obtenido.

Una decisión importante para esta primera versión fue la elección de las estructuras de datos a utilizar, ya que inciden en la complejidad final de la implementación. En esta primera versión, buscando en primer lugar comprender la lógica en sí del algoritmo y poder verificar su correctitud, se eligieron estructuras de datos simples y nativas, lo más similar posible a las del paper.

Π_s se implementó como `List<Set<Long>>`, es decir una lista de conjuntos de estados. Cada `Set<Long>` representa un bloque de estados. Π_t se implementó como `List<Set<Triple<Long, String, Long>>>`, es decir una lista de conjuntos de transiciones. Cada `Set<Triple<Long, String, Long>>` representa un bunch de transiciones no-inertes. HG ⟨Está muy bien todo esto, creo que quizá si modificas arriba los tipos de datos para hacerlos más pseudocódigo puedes hacerlo acá también⟩

A su vez, se debió utilizar una estructura `Pi_t cola` para poder iterar sobre los bunches a la vez que se actualizaba la lista de pendientes a refinar. Tal como se menciona en el paper de Groote y Jansen, se utiliza también la estructura del `splitterList` para almacenar los splitters generados durante el proceso de refinamiento. A su vez, se creó la clase `Splitter` para representar cada splitter, que contiene el bloque que se va a partir, el conjunto de transiciones que cruzan bloques y un flag de `primary/secondary`, un conjunto de transiciones marcadas para identificar que esos bloques son estables y un `groupId` para identificar qué Splitters se originaron a partir de la misma división inicial.

En primer lugar, el primer preprocesamiento requerido para este algoritmo es eliminar las componentes tau fuertemente conexas. La implementación de este paso usa la lógica del algoritmo de Kosaraju, que tiene una complejidad $O(n + m)$, por lo que cumple con la complejidad prometida por Groote y Jansen. Luego, se debe inicializar Π_s , que arranca con dos bloques (B_{vis} y B_{invis}), donde B_{vis} contiene el conjunto de estados desde los cuales se puede alcanzar una transición visible y B_{invis} es su complemento. Para realizar esta inicialización, se ejecuta la función `computeBvis` que genera un grafo de predecesores, almacenando para cada transición, cuál fue su origen. Luego, identifica cuáles son visibles y realiza un DFS para propagar las acciones visibles e identificar desde qué estados son

alcanzables. Por lo tanto, la complejidad resultante de la función es también $O(n + m)$. Finalmente, B_{vis} se inicializa con el conjunto de estados identificados por esta función como visibles y B_{invis} se inicializa con el resto. Π_t es inicializado con un único bunch que contiene todas las transiciones no-inertes.

Una de las primeras estructuras que también se deben inicializar en el preprocesamiento es el `stateToBlockMap`, definido como un mapa `Map<Long, Set<Long>>`, que almacena para cada estado a qué bloque de Π_s pertenece. Esta estructura es indispensable para poder realizar el seguimiento y las modificaciones rápidas que identifican a cada estado con su bloque, ya que la lógica principal del algoritmo se basa en modificar estos bloques continuamente. A su vez, esta estructura es acompañada por la función `updateStateToBlockMap(Map<Long, Set<Long>> map, Set<Long> newBlock1, Set<Long> newBlock2)` que recibe dos bloques nuevos y actualiza el mapa para que cada estado de cada bloque nuevo quede asociado a su bloque correspondiente.

Luego del preprocesamiento y la inicialización de las estructuras de datos, el código tiene un *loop* principal que itera sobre los bunches. En cada ciclo, se toma un bunch T . Si T es un bunch trivial, es decir que todas sus transiciones con la misma acción a van a un mismo bloque, entonces el bunch T no debe ser refinado. Si T no es trivial, entonces se buscan todos los "slices" de T , es decir, se buscan y se separan los subconjuntos de T que tienen la misma acción a y bloque de destino B' . Luego, se itera sobre todos los slices de T y se separa el primer slice encontrado que cumpla que es "chico", se lo identifica almacenándolo en la estructura `chosenTransitions`. Ahora, el objetivo es dividir a T , separando las `chosenTransitions` de T y dejando el resto de T en otra estructura que llamada `newBunch`. Luego, se agrega tanto el `chosenTransitions` como el `newBunch` a Π_t y a la cola de bunches por refinar. Ahora, se tienen dos bunches en Π_t que no son estables respecto a ambos bunches, por lo que se deben refinar los bloques de Π_s para que lo sean. Para esto, se utiliza la función `findSplittableBlocks`, donde se buscan todos los bloques que contienen transiciones en los dos nuevos bunches. Por cada bloque B encontrado, se crea un splitter primario y otro secundario y se guardan en una lista `splitterList`. El splitter primario contiene todas las transiciones de `chosenTransitions` que salen de B , y el splitter secundario contiene todas las transiciones de `newBunch` que salen de B . A su vez, se debe marcar como transiciones necesarias a reestabilizar todas las transiciones de `chosenTransitions`, es decir, las marcas del splitter primario. En esta primera versión, se utiliza un mapa `Map<Splitter, Set<Triple<Long, String, Long>>> markings` para almacenar estas marcas, asociando a cada splitter el conjunto de transiciones que fueron marcadas para la reestabilización. Por cada estado que es origen de una transición primaria, se debe identificar si ese estado también tiene una transición secundaria, y marcar a una transición secundaria por cada estado, añadiéndola a `markings`, asociadas al splitter secundario.

Luego, se itera sobre la `splitterList`. Por cada bloque B que se está observando para realizar la partición, se define el conjunto de estados R que son los estados de B que pueden alcanzar inertemente una transición del splitter y el conjunto de estados U que son los que no. Esta división en nuevos bloques R y U se hace a través de una función "split" que se detallará más adelante. Siguiendo la lógica del paper, si el splitter actual era primario, entonces luego del split, el bloque U es estable respecto al bunch `chosenTransitions`, por lo que se debe crear un nuevo splitter secundario para el bloque B , que solo contiene las transiciones de `chosenTransitions` que salen de R . Es decir, se reemplaza el splitter secundario de B por el nuevo splitter secundario para R . Al realizar

el split de R y U , las transiciones inertes salientes de R hacia U , antes eran inertes pues estaban dentro del bloque U , pero ahora es necesario identificarlas como no-inertes, ya que ahora cruzan dos bloques separados. Para realizar esta identificación, se utiliza la función `findNewNonInertTransitions`. Estas nuevas transiciones no-inertes, conforman un nuevo bunch que se debe agregar a Π_t . A su vez, tal como se menciona en el paper, R puede haberse vuelto inestable respecto al nuevo bunch creado. Por ello, se vuelve a utilizar la función `split` para volver a dividir R en dos bloques, N y R' . Si N y R' no son vacíos, entonces se reemplaza a R de Π_s por N y R' .

Acorde con el análisis hecho en el paper, la estabilidad no se preserva cuando aparecen nuevos estados bottom, por lo que se debe estabilizar al nuevo bloque N con respecto a todos los bunches a donde N tenga transiciones salientes. Para ello, se identifican todos los bottom states de N y se itera sobre Π_t para encontrar todos los bunches que contienen alguna transición saliente de N . Por cada bunch encontrado, se crea un splitter secundario. Por último, se itera sobre los estados bottom identificados de N y se marca a una transición por cada estado. Estas marcas se realizan agregando a la primera transición del estado en una lista de estados `marksForRestabilization`.

Retomando, la función `split` es el corazón del algoritmo. La función `split` es la que se encarga de dividir al bloque B en los bloques R y U , y la implementación de esta función es la que hace la diferencia en el paper para poder alcanzar la cota de la complejidad. En el paper se detalla a la función `split` como dos corutinas que se ejecutan en paralelo. Sin embargo, con vistas a nuestro objetivo de poder realizar primero una primera implementación del algoritmo y validar su correctitud, en esta primera versión no se implementó la función como dos corutinas. La función `split` tiene la siguiente firma:

```
private static Pair<Set<Long>, Set<Long>> split(
    Set<Long> B,
    Set<Triple<Long, String, Long>> transitions,
    Set<Triple<Long, String, Long>> currentMarks,
    MTS<Long, String> toMinimise,
    Set<String> tauLabels,
    Map<Long, Set<Long>> stateToSCCMap)
```

Es decir, la función `split` recibe como argumentos el bloque B a dividir, las transiciones del splitter asociado, las marcas del splitter asociado, el LTS a minimizar, el conjunto de etiquetas identificadas como tau y el mapa de estados a bloques. A su vez, `split` devuelve un par de conjuntos de estados, el bloque R y el bloque U . En esta versión, `split` se limita a calcular el conjunto R usando *backward reachability* para las transiciones inertes. Para comenzar este procesamiento, todos los estados con alguna transición marcada (es decir, incluida en `currentMarks`), son puestos en el nuevo bloque R . A su vez, se recorren todas las transiciones del splitter para incluir también en R a los estados desde los cuales estas transiciones salen. Luego, se construye el mapa `inertPredecessorsInB`, que almacena para cada estado cuáles son sus predecesores inertes. Luego, se propagan las transiciones inertes hacia atrás, utilizando una estructura `worklistR` que comienza con todos los estados de R , y por cada uno de ellos busca qué estados son predecesores inertes y los agrega a R y a la `worklist` para procesarlos también. El proceso continúa hasta que no queden predecesores inertes y por lo tanto, R sea estable. Luego, se devuelve el nuevo bloque R y el nuevo bloque U , definido como el complemento de R en B .

5.3. Primeros tests

Como se mencionó anteriormente, la estrategia fue desarrollar una primera versión fiel al algoritmo para entender la lógica y poder validar su correctitud. Por eso, luego de realizar esta primera implementación, se desarrollaron casos de tests para verificar los funcionamientos básicos del algoritmo. Se realizaron en primera instancia tests unitarios básicos, con casos desde 2 estados hasta 80 estados. Se creó un archivo de testeo unitario llamado `BranchingEquivalenceTest.java`, ubicado en `mtstools/DCS/Compositional/Tests/`, donde se implementó la lógica básica para a raíz de archivos `.lts` llamar al algoritmo de *branching bisimilarity* y luego al algoritmo de *weak equivalence* y comparar el grafo resultante obtenido. Para esto, primero se realizaron pruebas de todos los casos donde ambos algoritmos tenían que reducir al mismo gráfico, es decir, casos donde los algoritmos se comportaran de forma equivalente. Luego de validar estos casos, el objetivo fue encontrar un caso donde la lógica de los algoritmos difiera, es decir que por el caso de estudio del grafo elegido, ambas reducciones por la definición de equivalencia que utilizan, se comporten de forma diferente **HG** *(como?, quizá acá o en algún lado conviene mostrar este ejemplo donde un algoritmo debería reducir y el otro no)*. Para ello, se implementó el caso de test presentado por [ROB J. VAN GLABBEK y W. PETER WEIJLAND](https://dl.acm.org/doi/epdf/10.1145/233551.233556) en <https://dl.acm.org/doi/epdf/10.1145/233551.233556>. Este caso de test fue importante para poder asegurar que el algoritmo de *branching bisimilarity* realmente funcionara de la forma esperada, permitiendo validar de forma más robusta su implementación. A su vez, permitió desarrollar a un nivel más práctico las diferencias reales entre ambos algoritmos.

agregar cita

5.3.1. Integración con el algoritmo composicional

Luego de realizar los primeros tests, se revisó también la integración del nuevo algoritmo de *branching bisimilarity* con MTSA. En este contexto, se pudo detectar que en el contexto del algoritmo, una acción fluent se debería distinguir de las transiciones tau, y aun cuando esté identificada como local, no debe ser tratada como tau. Es por esto, que en la siguiente versión se agrega una validación para eliminar a las acciones fluents del conjunto de `tauLabels`.

Otra diferencia a como estaba implementado el algoritmo es que en MTSA el estado de error $-1L$ es particular. En el algoritmo inicial, esto no estaba contemplado, pero luego al realizar las primeras pruebas, lo modifiqué para que el estado de error tenga un bloque inicial propio.

A su vez, para poder integrar el algoritmo de minimización de *branching* en el algoritmo composicional, se creó la función `buildMinimisedMTSFromPartition`, que a partir de una partición que devuelve el algoritmo de *branching bisimilarity* se crea el MTS correspondiente.

5.4. Primeras optimizaciones

5.4.1. Cambio de estrategia: dos fases que alternan

Para la primera versión del algoritmo con optimizaciones, la mayor diferencia con la estructura del paper es que se implementan dos fases que alternan al mismo nivel en lugar de anidar un `while` externo sobre bunches con un `for` interno sobre splitters. Si bien este cambio no afecta la complejidad asintótica, mejora el entendimiento del algoritmo y

ayuda a introducir los cambios que mejoran el uso de las estructuras. A su vez, también permite tener dos fases separadas y poder utilizarlas como elemento de medición para contar iteraciones y tiempos. Entonces, se modifica el algoritmo para que esté compuesto por dos fases: la primera donde se realiza la estabilización de estados y la segunda donde se refinan los bunches correspondientes.

En la primera fase, se itera sobre la splitter list, que almacena una estructura por cada split pendiente, indicando que falta estabilizar el bloque B con el conjunto de transiciones asociadas y las marcas correspondientes. Por cada splitter, si el bloque no está en Π_s , quiere decir que ya fue partido por un splitter anterior y se descarta. Esto no pasará en la primera ronda pero puede suceder en las rondas futuras. Si no, se ejecuta el `split(B, transitions, marks)` que devuelve los bloques R y U al igual que en la primera versión. Al dividir a B en R y U, hay que actualizar los splitters relacionados a B. En la primera versión, esto era necesario solo para el splitter secundario, pero como ahora el algoritmo se divide en dos fases, entonces se define la función `refineSplitters` que recorre la `splitterList` y por cada splitter relacionado al bloque B, se crean dos splitters nuevos para R y U recalculando nuevamente las marcas necesarias y manteniendo el `groupId`. Estos splitters son añadidos a la `splitter list` para que sean procesados a continuación. Al igual que antes, el split no solo afecta a los splitters, sino que también puede hacer que un bunch trivial se vuelva no trivial luego de la partición. Para encolar a estos bunches, en esta nueva versión se implementa la estructura nueva `targetStateToBunches` que es de tipo `Map<Long, Set<Set<Triple>>>` y permite almacenar por cada bloque B cuáles son los bunches que tienen alguna transición entrante a B. Esto permite no iterar sobre todo Π_t para encontrar los bunches afectados y encolarlos fácilmente. Por último, también puede ser que haya transiciones inertes en el bloque b que al dividirlo ya no sean inertes. Este proceso se optimizó en esta versión para acercarse más a lo sugerido en el paper. Para ello, se utiliza una nueva estructura `newFrontiers` que es un `Queue<Pair<Set<Long>, Set<Long>>>`, donde cada elemento es un par que contiene un bloque en cada posición, por ejemplo `<R,U>` y `<U,R>`, indicando que se debe revisar si hay transiciones que vayan de R a U que anteriormente eran inertes pero ahora dejaron de serlo. Por cada par, se buscan estas transiciones y se registra como nuevo bunch en las estructuras de datos indicadas, como Π_t . Luego, al igual que en la versión inicial, se vuelve a partir el bloque R en N y R', pero utilizando los nuevos mecanismos detallados para esta versión. El invariante que se asegura al finalizar la Fase 1, es que Π_s es estable y la `splitterList` finaliza vacía.

La segunda fase toma un bunch no trivial T de Π_t . Para T, se inicializa un mapa `Map<Pair<String, Integer>, Set<Triple<Long, String, Long>>>` llamado `slices`, donde por cada par `<a, blockID>` se almacena el conjunto de transiciones de T que tienen acción a y entran al bloque `blockID`. Esta es otra optimización que se agrega en esta versión, ya que antes los bloques se representaban como un `Set<Long>` pero esto representaba un costo $O(n)$ cada vez que se buscaba un bloque. En esta nueva versión, se utiliza la estructura `blockIdMap` implementada como un `IdentityHashMap<Set<Long>, Integer>`. Esta estructura asocia por cada bloque un Integer, que actúa como su `blockID` asociado, permitiendo obtener el número de bloque en $O(1)$. Retomando el algoritmo, T es trivial cuando hay una sola slice, por lo que en ese caso si el bunch elegido tiene una sola slice, se toma otro bunch de Π_t hasta obtener uno que no sea trivial. Al encontrar un bunch T no trivial, es decir que tiene más de un slice, al igual que en la primera versión, se eligen las `chosenTransitions` y se realiza el mismo procedimiento que el mencionado

anteriormente para partir T en `chosenTransitions` y `newBunch`, también actualizando Π_t y los splitter asociados.

Entonces, la estrategia en esta nueva versión del algoritmo fue cambiar el orden de la lógica para primero procesar todos los splitters pendientes en la fase 1, luego realizar la partición de un bunch no trivial en la fase 2 y encolar nuevos splitters, para finalmente volver a iterar los splitters en la fase 2. Estas dos fases se ejecutan de manera alterna hasta que las estructuras Π_t y la `splitterList` se encuentran vacías, ya que eso asegura que todos los bloques son estables respecto a los conjuntos de transiciones, y que a su vez todas las transiciones son no triviales.

5.4.2. Implementación de nuevas estructuras

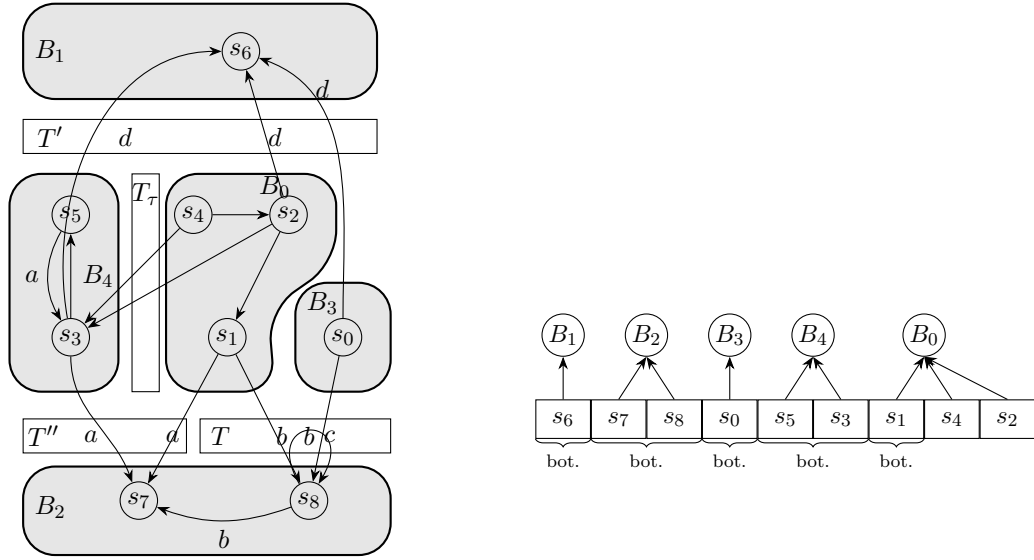
Con el objetivo de optimizar los tiempos del algoritmo, se reemplazaron varias estructuras poco ágiles de la primera versión. Por ejemplo, Π_t , antes implementada como un `List<Set<Triple>>`, ahora fue implementada como un `Set<Set<Triple>>` con `IdentityHashMap`. A su vez, en la clase `Splitter` también se realizaron varias modificaciones. Previamente, las marcas se marcaban en un mapa asociando el splitter con las marcas, ahora en cambio se implementa un campo `marks` dentro de la propia clase. Por otro lado, en la primera versión del algoritmo se explicó que para detectar cuando un splitter primario tenía un splitter secundario, se buscaba directamente el siguiente en la fila de splitters, asumiendo una adyacencia. Eso era válido en esa versión, pero ahora al tener dos fases distintas, se necesitó implementar dentro de la clase `Splitter` el campo `groupId`, marcando que si dos splitters tienen el mismo `groupId` están emparejados.

5.5. Alcanzando la complejidad teorica

Luego de realizar las primeras optimizaciones mencionadas y validar que el algoritmo funcionara correctamente, solo quedaba implementar dos cambios claves para alcanzar la complejidad teorica de $O(n \log m)$ tal como se detalla en el paper. En particular, el paper menciona dos estructuras claves en la sección "detalles de implementacion". Por un lado, detalla la implementacion de estados agrupados por bloques con la utilizacion de una estructura llamada *refinable partition*". Por otro lado, se menciona la implementacion de una estructura para almacenar los bunches de transiciones llamada "Linked Refinable Partitions.^a la que en el codigo final para esta tesis se decidio nombrar como "LinkedTransitionPartitions". Por ultimo, en esta nueva version se implementan tambien los cambios necesarios para poder ejecutar la funcion `split` como las dos corrutinas mencionadas en el paper.

5.5.1. Refinable Partitions

Las *refinable partitions* son una estructura de datos que reemplaza la forma de almacenar a Π_s como una lista `List<Set<Long>>`, la estructura `stateToBlockMap` y el id de bloque `blockIdMap` de las primeras versiones. La idea principal es que todos los estados viven en un único array `elements`. Cada bloque es un slice contiguo `[start..end)` de ese array. Por lo tanto, al separar un bloque no se necesita copiar nuevamente la estructura, sino que se reordena el array con swaps $O(1)$ y se agrega un nuevo slice de bloque. La Figura 5.1 (a) muestra un ejemplo de un LTS partido en 5 bloques, y la Figura 5.1 (b) hace un *snapshot* de cómo se vería la estructura de *Refinable Partition* para el LTS de



(a) Un ejemplo de LTS y su partición correspondiente. Las transiciones no etiquetadas deben asumirse como intertes, es decir τ -transitions.

(b) Estructura Refinable Partition para el LTS de ejemplo.

Fig. 5.1: Una refinable partition para un LTS de ejemplo. (a) El LTS y su partición en cinco bloques; las transiciones sin etiqueta se asumen τ (inertes). (b) El array maestro `elements`: cada bloque es un slice contiguo, y dentro de cada slice los estados bottom (marcados bot.) se ubican primero.

ejemplo, marcando también los slices contiguos de cada bloque. A su vez, dentro de cada bloque hay tres rangos disjuntos de estados, delimitados por dos punteros `bottomEnd` y `rDestEnd`. Esto se ve de forma poco clara pero introductoria en la Figura 5.1 (b), donde se puede observar que los estados no-bottom de B_0 y B_4 están agrupados de forma separada dentro de cada bloque, luego de los estados bottom, marcados como “bot.”. Una refinable partition tiene tres divisiones: primero en `[start..bottomEnd)`, se posicionan los estados bottom. Luego, en `[bottomEnd..rDestEnd)`, se almacenarán durante el algoritmo del split, los estados no-bottom que no van a estar en R . Por ultimo, posicionado al final de la estructura, en `[rDestEnd..end)`, se encontrarán los estados destinados a R . Se mantiene siempre el invariante `start ≤ bottomEnd ≤ rDestEnd ≤ end`.

La implementación de refinable partition se compone de dos clases: `RefinablePartition` y `Block`. La Figura 5.2 detalla la clase `Block`, compuesta por el campo de `start`, `end`, `bottomEnd`, `rDestEnd`, el `id` y si el bloque sigue activo, con `alive`. A su vez, expone también el método `size`.

A su vez, se define la clase `RefinablePartition`, detallada en la Figura 5.3. Se exponen los métodos de construcción, `RefinablePartition(Collection<Long> universe)` y `void seedFromInitialBlocks(List<Set<Long>> initialBlocks)`. Luego, los métodos de lookup, donde se pueden notar claramente las mejoras en los tiempos de acceso en esta estructura. `Block blockOf(long s)` devuelve el bloque al que pertenece el estado s , en $O(1)$. `Iterable<Long> states(Block b)` retorna los estados del bloque b , en $O(|b|)$. `Iterable<Long> bottoms(Block b)` retorna los estados bottoms del bloque b , en $O(|cantestadosbottomdeb|)$. `Iterable<Long> rDestStates(Block b)` retorna todos los estados destinados a R del bloque B , utilizando el puntero ya mencionado, `rDestEnd`, en

```

public static final class Block {
    int start;
    int end;
    int bottomEnd;
    int rDestEnd;
    public final int id;
    boolean alive;

    public int size()          { return end - start; }
}

```

Fig. 5.2: Definicion de la clase Block

$O(|b.rDestCount|)$. Por ultimo, `List<Block> liveBlocks()` retorna todos los bloques existentes en la `RefinablePartition`. Luego, se exponen tambien los metodos necesarios para realizar un split.

```

final class RefinablePartition {
    RefinablePartition(Collection<Long> universe);
    void seedFromInitialBlocks(List<Set<Long>> initialBlocks);

    Block blockOf(long s);
    Iterable<Long> states(Block b);
    Iterable<Long> bottoms(Block b);
    Iterable<Long> rDestStates(Block b);
    List<Block> liveBlocks();

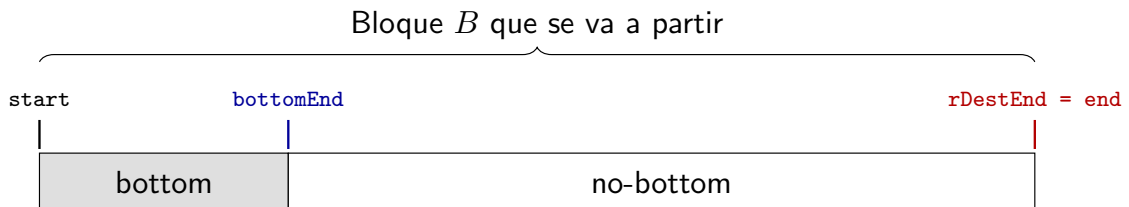
    void addToR(long s);
    void clearR(Block b);
    Pair<Block, Block> splitOffR(Block b);

    void markAsBottom(long s);
    void resetBottoms(Block b);
}

```

Fig. 5.3: Definicion de la clase RefinablePartition

La Figura 5.4 sigue cómo va mutando la estructura durante un split, paso a paso. Observar este proceso ayudara a explicar como funcionan los metodos `addToR`, `clearR` y `splitOffR` que son el corazon de la `RefinablePartition`. Observemos el slice de la estructura `RefinablePartition` que representa al bloque `B`. Para ejecutar las operaciones, lo único que se mueve son los estados dentro del slice de `B` (a traves de swaps) y los punteros `bottomEnd` y `rDestEnd`. En ningún momento se copian estados a una estructura nueva ni a otro slice. A continuacion, el paso a paso de un split.

Fig. 5.4: Paso (a): estado inicial del bloque *B* (`clearR`).

(a) `clearR(B)` (Figura 5.4). Por default, un bloque comienza con `rDestEnd = end`, por

lo que la particion destinada a R queda vacío. El bloque contiene sólo dos rangos visibles: los estados bottom [`start..bottomEnd`) y el resto no-bottom. La función `clearR` asegura que la particion de R comience vacía.

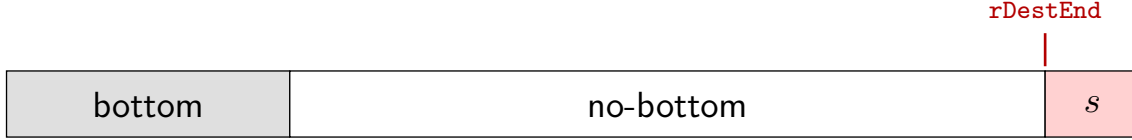


Fig. 5.5: Paso (b): `addToR` sobre un estado no-bottom.

(b) `addToR(s)` **sobre un no-bottom** (Figura 5.5). Si se debe agregar un estado s no-bottom a R , se hace un swap del estado contra la posición `rDestEnd-1` y se retrocede `rDestEnd` en uno. La particion R crece de a un estado por llamada, sin tocar los estados bottom. $O(1)$ ya que solo se debe realizar el swap y sumar o restar a los punteros.

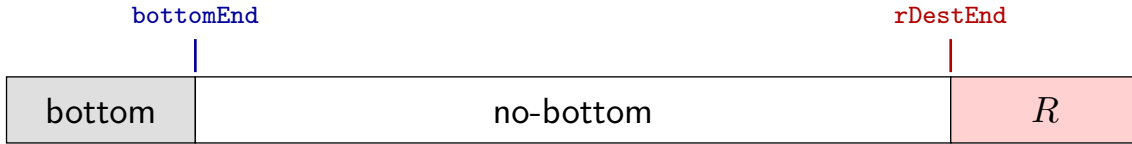


Fig. 5.6: Paso (c): `addToR` sobre un estado bottom.

(c) `addToR(b)` **sobre un bottom** (Figura 5.6). Si el estado era bottom, primero se lo saca de ahí con un swap contra `bottomEnd-1` y se retrocede `bottomEnd`. De esta forma, el estado queda incluido en la particion R al igual que en (b). $O(1)$ ya que solo se debe realizar el swap y sumar o restar a los punteros.

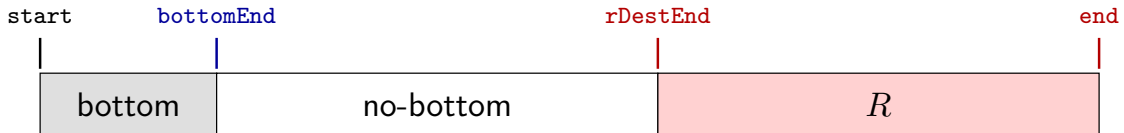


Fig. 5.7: Paso (d): estado del bloque tras repetir `addToR`.

(d) **Estado tras repetir `addToR`** (Figura 5.7). Una vez agregados todos los estados de R , la particion [`rDestEnd..end`) contiene los estados indicados y el slice de B quedó ordenado en tres particiones.

(e) `splitOffR(B)` (Figura 5.8). Se corta el slice en `rDestEnd`: la primera particion [`start..rDestEnd`) pasa a ser `uBlock` y la segunda particion [`rDestEnd..end`) pasa a ser `rBlock`, cada uno con su nuevo id. El viejo B se marca como muerto y se reasigna `blockOfElement` únicamente en las posiciones del sufijo para que apunten a `rBlock`. Es la única pasada lineal del split y reemplaza a la copia de dos `HashSet` de las versiones anteriores. $O(1)$ ya que lo unico que se hace es separar a R como otro bloque. Esto implica crear una nueva estructura `Block` con los punteros de `start` y `end`, pero no implica copiar todos los elementos de R .

(f) `findBottomStates` (Figura 5.9). Como al hacer el split los bottoms del bloque viejo ya no valen, se recomputan utilizando `resetBottoms`, que realiza `bottomEnd = start` y `markAsBottom` que realiza un swap y mueve el puntero `bottomEnd` segun corresponda.

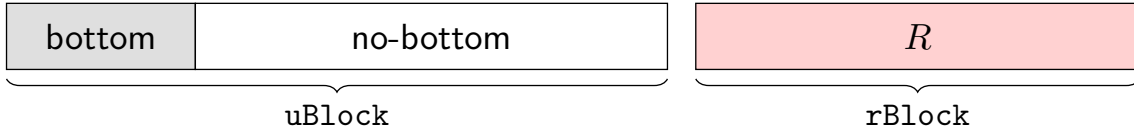


Fig. 5.8: Paso (e): `splitOffR` divide el bloque en `uBlock` y `rBlock`.



Fig. 5.9: Paso (f): `findBottomStates` recomputa los estados `bottom` en cada bloque nuevo.

Para concluir, tambien se reemplazo la inicializacion de Π_s , que en las anteriores versiones era implementado como una lista `List<Set<Long>>`, por las llamadas a los constructores de `RefinablePartitions`, tal como se ve en la Figura 5.10. Asi, se logra bajar la complejidad de varias operaciones frecuentes de Π_s . Por ejemplo, cuando previamente se queria ver si Π_s contenia un bloque `B`, se debia recorrer toda la lista de bloques, es decir que tenia complejidad $O(|\Pi_s|)$. Con la nueva estructura, esto se realiza en $O(1)$, simplemente viendo si el bloque `B` tiene el campo `alive` en `true`. A su vez, para encontrar los `bottom` states de un bloque `B`, se iteraba por todos los estados hasta encontrar los `bottom`. Ahora, se pueden encontrar todos los `bottoms` directamente en la particion `[start..bottomEnd)`.

```
RefinablePartition Pi_s = new RefinablePartition(toMinimise.getStates());
Pi_s.seedFromInitialBlocks(initialBlocks);
```

Fig. 5.10: Inicializacion de Π_s como `RefinablePartition`

5.5.2. Linked Transition Partitions

En el paper de Jansen tambien se sugiere la implementacion de Π_t como "linked refinable partitions". Asi, la idea es reemplazar los bunches de Π_t que en las primeras versiones estaban representados como `Set<Triple<Long, String, Long>>`. Para ello, se define la clase `Transition` tal como muestra la Figura 5.11, donde cada transicion es un objeto que almacena el estado de origen, la accion y el estado destino. A su vez, se almacena el bunch de Π_t que la contiene, y el block-bunch al que pertenece, es decir el subgrupo que indica dentro de la bunch a la que pertenece, de que estado de origen es. Por ultimo, se implementa el campo `untested`, necesario para la implementacion de las corrutinas en el `split`.

Luego, se define la clase `LinkedTransitionPartitions`, tal como se ve en la Figura 5.12. Se hace referencia a Π_s ya que se debe consultar en diversos metodos, debido a que los bunches dependen de los bloques de estados. La cola de trabajo El campo `globalUnstable` es la lista de todas las block-bunch-slices inestables, es decir, la cola de trabajo pendiente de la Fase 2: implementada como una lista con position tracking, permite agregar y quitar elementos en $O(1)$. Por ultimo, `nextBunchId` y `nextBlockBunchId` son contadores que asignan un identificador unico a cada `BunchSlice` y `BlockBunchSlice` en el momento de su creacion. A su vez, esta la lista `allTransitions`, que es la lista que contiene todas las transiciones del LTS. Esta es la lista principal sobre la cual se construyen las cuatro vistas de las transiciones descritas en el paper de Jansen que permiten

```

public static final class Transition {
    public final long source;
    public final String action;
    public final long target;
    BunchSlice bunch;
    BlockBunchSlice blockBunch;
    int untested;
}

```

Fig. 5.11: Definición de la clase Transition

sostener la complejidad deseada.

```

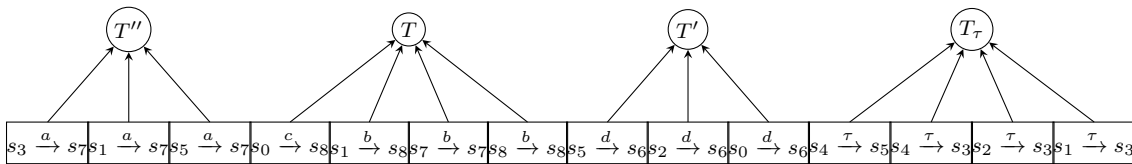
public final class LinkedTransitionPartitions {
    RefinablePartition Pi_s;
    List<Transition> allTransitions;
    Map<Long, List<Transition>> bySource;
    Map<Long, List<Transition>> byTarget;
    List<BunchSlice> liveBunches;
    Map<RefinablePartition.Block, List<BlockBunchSlice>> slicesByBlock;
    List<BlockBunchSlice> globalUnstable;
    Map<Triple<Long, String, Long>, Transition> transitionLookup;

    nextBunchId;
    nextBlockBunchId;
}

```

Fig. 5.12: Definición de la clase LinkedTransitionPartitions

Por un lado, esta la vista por bunch `liveBunches`. Esta vista se implementa como una lista de bunch slices `List<BunchSlice>`, donde la clase `BunchSlice` agrupa las transiciones de un mismo bunch de Π_t . Esta es la vista que implementa directamente la partición Π_t , y reemplaza al `IdentityHashMap` que en las versiones anteriores representaba el conjunto de bunches. La Figura 5.13 la muestra como queda la estructura sobre el ejemplo de la Figura 5.1. A diferencia del paper, esta vista no mantiene de forma explícita los action-block-slices (los subconjuntos de transiciones de un mismo bunch con igual acción y bloque destino), sino que estos se calculan momentáneamente solo cuando hace falta refinar un bunch, en el método `peelSlice`.

Fig. 5.13: La estructura de `liveBunches`, que representa la vista de todas las transiciones ordenadas por bunches de Π_t . Se usa el ejemplo de la Figura 5.1.

Luego, la segunda vista es `slicesByBlock`, que se implementa como un mapa `Map<Block, List<BlockBunchSlice>>`, donde cada bloque de Π_s tiene asociada una lista de todos sus block-bunch-slices, que serían las transiciones que se originan desde bloque dentro de cada bunch. Este mapa permite recuperar en $O(1)$ todas las block-bunch-slices de un bloque, lo cual es utilizado cuando se parte un bloque, ya que se debe redistribuir las block-bunch-slices del bloque viejo entre los dos bloques nuevos. La Figura 5.14 muestra la estructura

de `slicesByBlock` para el ejemplo presentado anteriormente.

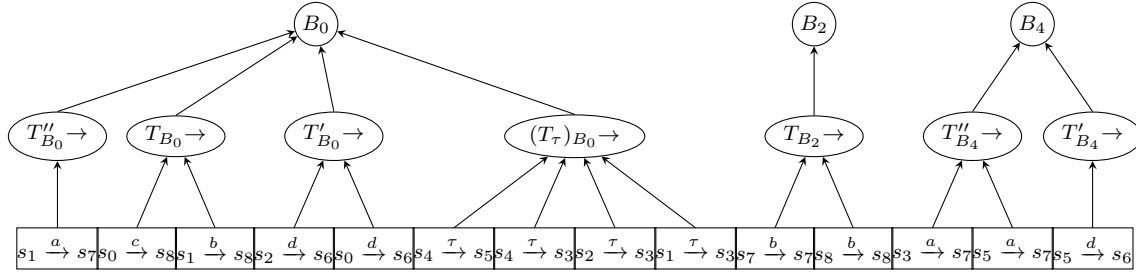


Fig. 5.14: La estructura de `slicesByBlock`, mostrando la vista de las transiciones agrupadas por block-bunch-slice.

La tercera vista mencionada agrupa las transiciones por su estado de origen. Se implementa como un mapa de tipo `Map<Long, List<Transition>>` llamado `bySource` y se expone con el metodo `outgoing(s)`, que devuelve todas las transiciones salientes del estado s . Esta vista es estatica, ya que como la lista de transiciones salientes de un estado se fija segun el LTS de entrada y no varia, la estructura no se modifica en ningun paso del algoritmo. Se usa de referencia para poder obtener todas las transiciones salientes de un estado, por ejemplo, durante el split. La Figura 5.15 muestra la estructura, donde cada estado de origen tiene asociada la lista de sus transiciones salientes.

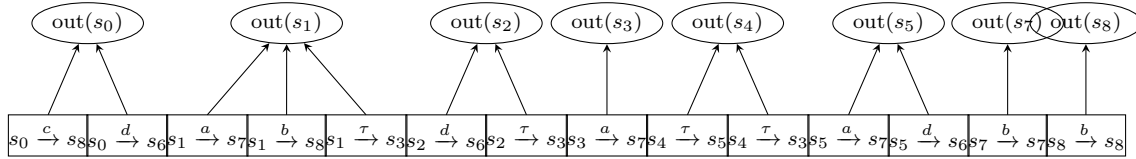


Fig. 5.15: La estructura (`bySource`), que representa la vista de transiciones ordenadas por estado de origen.

Por ultimo, la cuarta vista es la otra de la anterior y agrupa las transiciones por su estado destino, tal como muestra la Figura 5.16. Se implementa como un mapa de transiciones `Map<Long, List<Transition>>` denotado `byTarget` y se expone con el metodo `incoming(s)`, que devuelve las transiciones entrantes al estado s . Por el mismo motivo que la vista anterior, tambien es estatica. Optimiza de forma el proceso de cuando un bloque se parte, ya que se deben marcar como inestables las block-bunch-slices de los bunches que tenian transiciones apuntando a los estados del bloque que se separo, y `byTarget` permite acceder a ellas facilmente recorriendo para cada estado del nuevo bloque, sus transiciones entrantes.

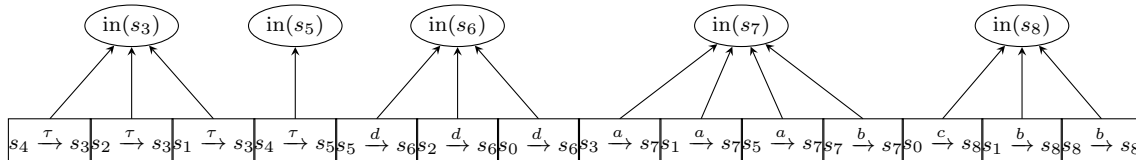


Fig. 5.16: La estructura `byTarget`, mostrando las transiciones agrupadas por estado destino.

Es importante aclarar que las cuatro vistas no implica que haya cuatro copias de las transiciones, sino que existen cuatro agrupaciones de los mismos objetos `Transition`,

ya que cada transición guarda punteros a su `BunchSlice` y a su `BlockBunchSlice`, y el puntero a la propia `Transition` es lo que se relaciona en las listas de `bySource` y `byTarget`. Es por esto que el paper las llama "linked", ya que una única instancia participa de las cuatro vistas/particiones a la vez.

5.5.3. El método split como corrutinas

Como se explicó anteriormente, la función `split` en el paper de Jansen se idea originalmente como dos corrutinas, donde la corrutina de `R` comienza con los estados iniciales que deben ir en `R` y luego extiende el conjunto hacia atrás, recorriendo las transiciones inertes. A su vez, la corrutina de `U` identifica a los estados que no están en `R`, contando por cada estado cuántos son sus sucesores que todavía no están en `U`. Ejecutar estas corrutinas de forma alternada, un paso cada una (en *lockstep*), es la esencia de la complejidad del paper, ya que en cuanto una corrutina agrega a su conjunto más de la mitad de estados totales que tenía el bloque, se corta su ejecución. Este corte consigue que cada llamada a la función `split` pase de tener que recorrer todo el bloque en peor caso, a que pase a recorrer solo el subbloque más chico de los dos conjuntos de estados resultantes. Sin embargo, en las primeras versiones la implementación se había reducido a realizar *backwards-reachability* para `R`, es decir, no se había desarrollado la optimización sugerida en el paper. Por su importancia en la complejidad final, era esencial implementarla en nuestra versión en MTSA. Sin embargo, Java no tiene la posibilidad de implementar corrutinas de forma nativa.

Para poder subsanar esa dificultad en la implementación, se implementaron dos funciones: `rStep` que ejecuta un paso en el análisis del bloque `R` y `uStep` que ejecuta un paso en el análisis del bloque `U`. Ahora, ¿cómo representamos qué es un "paso" en las funciones? Para esta implementación, un paso en cada función es analizar el próximo estado que corresponde chequear si pertenece al subbloque de la función. Para ello, se implementaron dos estructuras de colas, una para cada función, que contienen los estados que faltan analizar. En cada paso, cada corrutina toma el siguiente paso y ejecuta el análisis correspondiente para ver si ese estado corresponde o no en su conjunto. A su vez, se implementan los contadores "`rSize`" y "`uSize`". Esto permite validar que cuando alguno de los dos contadores supere la mitad de los estados del bloque, se deje de ejecutar esa corrutina y no se sigan analizando estados para ese subbloque, haciendo que solo finalice la corrutina del subbloque más chico y luego se calcule el conjunto final de estados para cada subbloque, tal como se detalla en el paper y tal como fue explicado en la sección previa.

Si bien esta implementación no mantiene de manera completamente fiel el detalle del paper, permite que repliquemos la cota de la complejidad dentro de las limitaciones del lenguaje utilizado.

6. VALIDACIÓN Y EXPERIMENTACIÓN

6.1. Metodología

Luego de finalizar la implementación, continuamos con la evaluación del código. Por un lado, se quería evaluar la *correctitud* de la versión final del código del algoritmo en MTSA, y por el otro lado se quería evaluar el *rendimiento* del mismo, para luego poder compararlo con el algoritmo de WSOE previamente implementado en MTSA.

Al comenzar a realizar estas experimentaciones, se intentó reemplazar el llamado al algoritmo de WSOE dentro del proceso de síntesis composicional, por el llamado al recién implementado algoritmo *Branching bisimilarity*. Sin embargo, a medida que se comenzaron a realizar estas experimentaciones, la obtención de resultados se hacía cada vez más costosa y poco escalable, ya que el tiempo que tardaba en correr el proceso de síntesis completo era muy largo, lo que no nos permitía tener flexibilidad a la hora de afinar y mejorar los resultados obtenidos. Por ello, se aplicó una metodología de captura de las instancias de minimización, de forma que se corrió una vez el algoritmo de síntesis composicional completo con los conjuntos de casos de prueba ya armados por Uchitel y Gagliardi y en el contexto de este proceso, se almacenó en diferentes archivos todos los parámetros que eran introducidos en el llamado al algoritmo de minimización. Esto nos permitió extraer 2212 instancias de casos de LTS a minimizar, con las que luego pudimos validar y experimentar de forma más flexible y eficiente con las diferentes versiones de nuestra implementación del algoritmo de *Branching bisimilarity*.

Cada instancia capturada queda representada por un par de archivos. El primer archivo, de tipo `.lts`, contiene el autómata a minimizar en el formato FSP que interpreta MTSA, es decir, sus estados y sus transiciones etiquetadas. El segundo archivo almacena en formato JSON el resto de los parámetros que la llamada a la minimización recibe junto con el LTS, lo que incluye principalmente el alfabeto local, las etiquetas controlables y las locales. Luego, cada par `.lts/.json` contiene toda la información necesaria para reconstruir exactamente el problema de minimización de forma aislada, sin necesidad de reejecutar la síntesis composicional que lo generó.

Cada corrida de experimentación para un algoritmo de minimización registra sus resultados en un archivo CSV en el que se agrega una fila por cada instancia, versión del algoritmo y repetición, para permitir correr varias veces cada instancia y luego tomar la mediana de los valores resultantes. Además del tamaño del resultado (cantidad de estados, de transiciones y de transiciones *tau* del LTS minimizado), cada fila guarda el tiempo de ejecución y el uso de memoria. Para obtener mediciones más confiables, cada instancia se ejecutó 3 veces, y se registró también el estado final de la corrida (éxito, error o *timeout*).

Por último, se corrió la misma para tres versiones del nuevo algoritmo de *Branching bisimilarity* implementado en MTSA. En primer lugar, la v1 que es la primera implementación *naive* del algoritmo, presentada en la sección 5.2. Luego la v2, que es la presentada en la sección 5.4, con las primeras optimizaciones. Luego, la v3, que es la versión final del código con todas las optimizaciones ya implementadas, tal como se detalla en la sección 5.5. Por último, se realizó a su vez una corrida de la experimentación del algoritmo de WSOE para poder utilizar sus resultados en comparación.

Los experimentos fueron ejecutados en una computadora con procesador Intel i7-7700

agregar ref

agregar ref

agregar ref

CPU @ 3.60GHz, con 8GB de RAM.

6.2. Validación de correctitud

Con el objetivo de validar la correctitud del algoritmo, se buscó validar en primer lugar tomar instancias de diversos tamaños y de distintos casos, asegurando construcciones variadas, y se utilizaron a estas instancias para comparar el resultado obtenido con el algoritmo de *branching* con el resultado obtenido por la herramienta mCRL2, desarrollada en el contexto del paper de Jansen. Esto nos permitió validar la correctitud de nuestro algoritmo como primer paso.

Luego, se corrió también una validación de correctitud más fuerte, corriendo el algoritmo de WSOE y el de *Branching* y validando que para cada instancia, nunca debía suceder que *Branching* junte dos estados que WSOE dejaba separados, ya que eso implicaría un bug en el código desarrollado, tal como se detalló en la sección de teoremas .

Luego de validar la correctitud y asegurar que el algoritmo estaba funcionando en términos teóricos tal como se esperaba, se procedió con la experimentación.

6.3. Resultados

Tal como se detalló anteriormente, las instancias fueron capturadas replicando los casos de prueba del procesamiento de síntesis de controladores. En este contexto, la mayor parte de las ocasiones se busca minimizar grafos lo más pequeños posibles, y son pocas las ocasiones donde se debe minimizar un grafo más grande. Es por eso que el 14,6 % de las instancias (322 de las 2212) tienen más de 100 estados. Los experimentos resultan más interesantes de analizar en casos donde los grafos son más grandes ya que ahí podremos ver mejor la materialización de la complejidad teórica. A su vez, la proporción entre estados y transiciones es desigual, ya que la mayoría de las instancias tiene más del doble de transiciones que de estados, lo que resulta en que más del 27 % de las instancias analizadas supera las 200 transiciones. Esto nos dará un enfoque particularmente interesante de analizar, ya que la complejidad de nuestro algoritmo está atada también al número de transiciones del LTS a minimizar.

6.3.1. Tamaño de la reducción final

En este contexto, nos interesaba analizar cómo impactaban para estas instancias el resultado final de la minimización realizada. En ese sentido, luego de correr la experimentación para WSOE y para la versión final de *branching* (v3), pudimos ver que en el 83 % de los casos analizados, el resultado de la minimización de WSOE fue el mismo que el obtenido por el algoritmo de *branching bisimilarity*. Es decir, solo en un 17 % de los casos sucedió el caso ejemplificado en el capítulo . Este resultado es útil para demostrar que en la mayoría de los casos, a pesar de la diferencia en la equivalencia teórica entre ambos algoritmos, los casos donde esta diferencia se materializa en la experimentación es la minoría.

6.3.2. Tiempos

Las Figuras 6.1a y 6.1b muestran el tiempo de minimización en función de la cantidad de estados iniciales, la primera en escala log-log y la segunda en escala lineal. En la escala

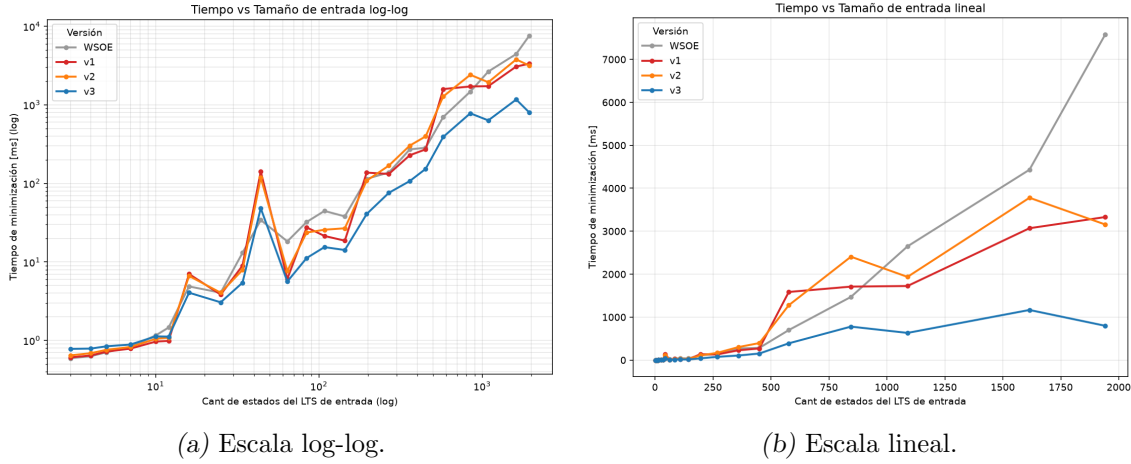


Fig. 6.1: Tiempo de minimización en función de la cantidad de estados del LTS de entrada.

log-log se ve que para instancias chicas el tiempo se ve dominado por el costo fijo de preparar la ejecución y no el algoritmo en sí, por lo que ahí no hay diferencias entre WSOE y las versiones de *branching*. Recién a partir de los $n \approx 100$ estados las curvas se separan y se vuelve visible el comportamiento asintótico de cada algoritmo, con v3 quedando consistentemente por debajo del resto. La escala lineal muestra de forma evidente la magnitud de esa diferencia en los grafos grandes, ya que mientras WSOE crece de forma prácticamente lineal con el tamaño y llega a tardar del orden de 7600 ms en las instancias más grandes, v3 se mantiene por debajo de los 1000 ms. Las versiones v1 y v2 siguen de cerca a WSOE, por momentos comportándose brevemente mejor. Esto demuestra que las optimizaciones finales sugeridas por el paper fueron claves para materializar la mejora asintótica en la complejidad del algoritmo.

Como se mencionó al comienzo de este capítulo, la mayoría de las instancias tiene muchas más transiciones que estados, y la complejidad teórica del algoritmo de *branching* depende justamente de la cantidad de transiciones. Por eso resulta interesante analizar el tiempo también en función de las transiciones. Las Figuras 6.2a y 6.2b muestran el tiempo de minimización contra la cantidad total de transiciones m del LTS de entrada (izquierda) y contra la cantidad de transiciones τ , ambas en escala log-log. En la Figura 6.2a se ve que v3 se mantiene por debajo de v2 y de WSOE a lo largo de casi todas las instancias, y que el tiempo de las versiones de *branching* queda correlacionado con m . La Figura 6.2b analiza particularmente las transiciones τ (excluyendo las instancias sin ninguna transición τ). Esto resulta interesante ya que por la naturaleza del algoritmo de *branching*, el algoritmo se debería comportar mejor para casos donde hay más estados posibles de minimizar, ya que allí se deben realizar menos iteraciones sobre las particiones. Este gráfico muestra justamente que la ventaja del algoritmo de *branching* se expone justamente sobre las instancias con más transiciones τ . Por último, nuevamente podemos ver como se muestra la mejora sobre los tiempos al implementar las optimizaciones mencionadas en el paper, que fueron claves para alcanzar y mejorar los tiempos de WSOE.

6.3.3. Memoria

Según el paper de Jansen, la utilización del espacio de memoria del algoritmo de *Branching bisimilarity* es menor a varios algoritmos previos con los que los autores la comparan.

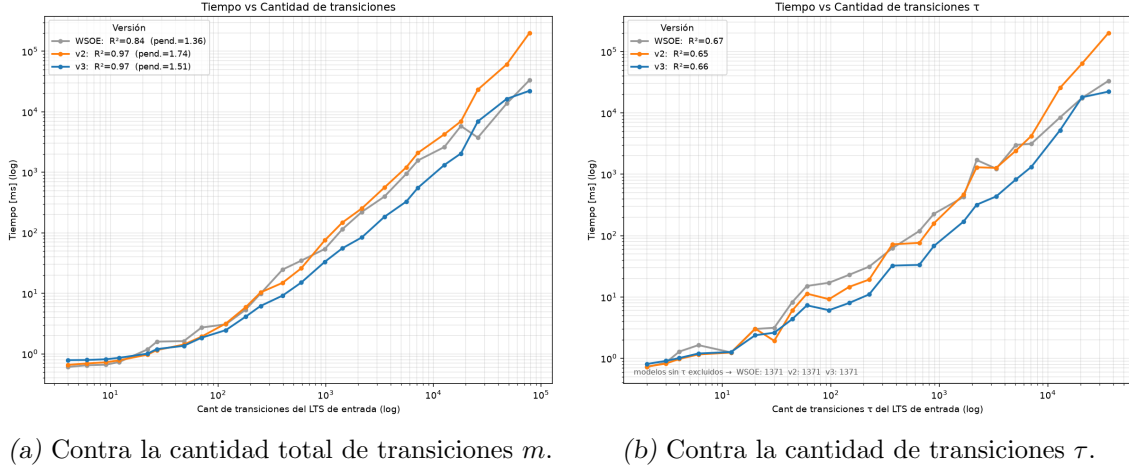
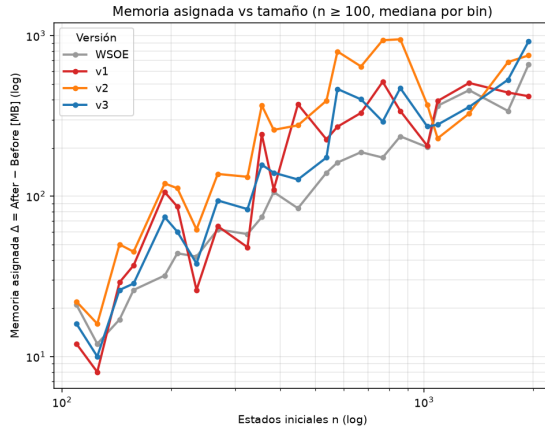
(a) Contra la cantidad total de transiciones m .(b) Contra la cantidad de transiciones τ .

Fig. 6.2: Tiempo de minimización en función de la cantidad de transiciones del LTS de entrada (mediana por bin de tamaño, escala log-log).

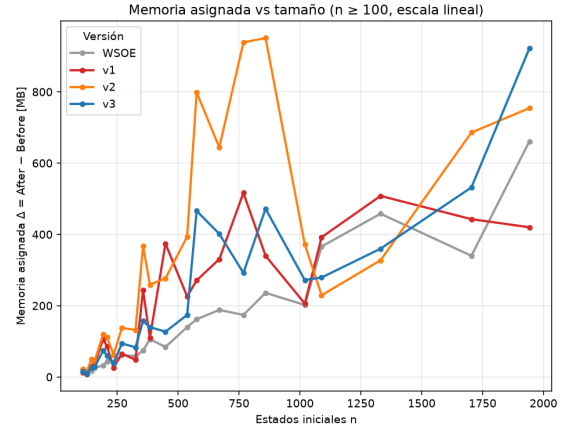
Sin embargo, a partir de las experimentaciones realizadas obtuvimos el resultado graficado en la Figura 6.3a. Como se muestra, todas las versiones de *Branching* utilizaron, en promedio, más memoria que la versión de WSOE. Para analizar este resultado, usamos la memoria *asignada* por cada minimización, $\Delta = \text{MemAfter} - \text{MemBefore}$. Para la mayoría de las instancias, este número se mantiene estable para todas las repeticiones de las corridas del algoritmo analizado, lo que nos da la pauta de que es una medida confiable.

Analizando la Figura 6.3a podemos ver que la v2 utiliza en casi todos los casos más memoria que las demás versiones de *Branching* y de WSOE. Analizando esta diferencia, se puede suponer que esto se debe a que la v2 es la que menos estructuras nativas de Java utiliza. Esta versión utiliza varios índices propios para introducir las primeras optimizaciones, como el IdentityHashMap el targetStateToBunches y la IdentityQueue, manteniendo a su vez la implementación de v1 de los bloques con Set<Int> y para los bunches, Set<Triple>. Estos índices duplican las referencias a los estados y a las transiciones, y por eso aumenta tanto el uso de la memoria. A su vez, v1 al no tener estas referencias, es más liviana. Por último, v3 vuelve a ser más liviana porque al introducir las clases de RefinablePartition y LinkedTransitionPartitions, el costo de los índices se absorbe dentro de la misma estructura.

Sin embargo, la comparación que más nos interesa es por qué el uso de memoria de v3 es mayor que el de WSOE. Analizando ambos algoritmos, podemos ver que WSOE utiliza una única estructura de datos para la partición de estados, por lo que no se debe mantener también nuevas estructuras para las particiones de transiciones, sino que eso se recalcula constantemente a partir de cada unificación de estados. Esto implica mayor tiempo de procesamiento pero también menos uso de memoria. Por último, también se debe considerar que el algoritmo de *Branching* declara varias estructuras de datos nuevas, propias del algoritmo, mientras que WSOE utiliza tipos de datos nativos para las estructuras. Esto resulta en que la recolección y optimización de memoria de Java sea más eficiente para WSOE que para *Branching*.



(a) Escala log-log.



(b) Escala lineal.

Fig. 6.3: Memoria asignada por la minimización ($\Delta = \text{MemAfter} - \text{MemBefore}$, mediana por bin de tamaño) en función de la cantidad de estados del LTS de entrada, para instancias con $n \geq 100$.

6.4. Discusión

7. CONCLUSIONES

A lo largo de este trabajo buscamos analizar la forma en la que el algoritmo DCS permite ser conceptualizado en una serie de *fases*, de las cuales comprendimos su funcionamiento durante la ejecución del algoritmo general de forma agnóstica a la heurística utilizada. Mediante la identificación de estas fases pudimos ver que el algoritmo (DCS) atraviesa etapas cuyos objetivos parciales no siempre se mantienen, más allá de ser etapas que conducen a un objetivo en común (resolver el problema de control en general). Por ejemplo, para encontrar una solución, primero es necesario encontrar un estado marcado y luego encontrar la manera de llegar de forma controlable a él.

Para llegar a ese análisis, primero atravesamos un proceso exploratorio sobre el algoritmo estándar y parte de su implementación para finalmente acceder al algoritmo *on-the-fly* y sus detalles. Este último, sumado a los detalles de la *heurística RA* fueron estudiados con precisión para poder desarrollar este trabajo.

Con este enfoque en mente y el trabajo realizado hemos logrado los siguientes objetivos:

- Definir y entender en profundidad las etapas que atraviesa el algoritmo (DCS) durante su ejecución y cuantificar el rendimiento de las heurísticas actuales para cada fase.
- Introducción de cambios a la heurística “Ready Abstraction” que impactaron en una de las fases de ejecución cuyo rendimiento es menor en términos relativos.
- Implementación de los cambios sobre la heurística incorporados en la herramienta MTSA [8]
- Mejora del rendimiento de la heurística con los cambios introducidos, tomando los casos del *benchmark* con ventana de mejora en la etapa 3.

Como trabajo a futuro, presentamos las siguientes ideas:

- Modificación del algoritmo de exploración *on-the-fly* para resolución de otros problemas, por ejemplo cambiando el objetivo de nonblocking por objetivos de tipo GR1.
- Análisis más profundo del impacto (positivo o negativo) que tienen los cambios propuestos en la exploración.
- Propuesta de nuevas modificaciones más refinadas a RA basándonos en el análisis de fases.

8. APÉNDICE

Bibliografía

- [1] D. Ciolek. Síntesis dirigida de controladores para sistemas de eventos discretos. PhD thesis, Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software (LaFHIS), FCEyN, UBA ,2018.
- [2] M. Durán, F. Zanollo. Síntesis dirigida de controladores para requerimientos de tipo non-blocking. Tesis de Licenciatura, Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software (LaFHIS), FCEyN, UBA, 2021.
- [3] Julian Braier, Daniel Ciolek, Matias Duran, Florencia Zanollo, Victor Braberman, Nicolas D’Ippolito, Sebastian Uchitel, Nicolas Pazos. *On-the-fly Informed Search of Non-blocking Directed Controller*.
- [4] S. A. Zudaire, M. Garrett, and S. Uchitel. Iterator-based temporal logic task planning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11472–11478, 2020.
- [5] P. J. Ramadge and W. M. Wonham. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization*, 25, 1987.
- [6] A. Pnueli and R. Rosner. On the Synthesis of a Reactive Module. In *Proc. of the Symp. on Principles of Programming Languages, POPL*, pages 179–190, 1989.
- [7] Dana Nau, Malik Ghallab, and Paolo Traverso. *Automated Planning: Theory & Practice*. Morgan Kaufmann Publishers Inc., 2004.
- [8] Modal Transition System Analyser (MTSA) <http://mtsa.dc.uba.ar/>
- [9] Jan Friso Groote, David N. Jansen, Jeroen J. A. Keiren, Anton J. Wijs. *An $O(m \log n)$ Algorithm for Computing Stuttering Equivalence and Branching Bisimulation*. ACM Transactions on Computational Logic, 2017. (versión extendida: arXiv:1909.10824, 2019.)
- [10] S. Mohajerani, R. Malik, M. Fabian. *A Framework for Compositional Synthesis of Modular Nonblocking Supervisors*. Chalmers University of Technology. https://publications.lib.chalmers.se/records/fulltext/165507/local_165507.pdf
- [11] L. Pousseur, S. Uchitel. *Compositional synthesis of discrete event controllers*. arXiv:2506.16557, 2025.